Project overview

-The Problem:

-Rapid growth in online transactions has led to sophisticated financial fraud. Traditional security systems fail to adapt, causing massive financial losses and harming customer trust.

-The Objective:

-To build an intelligent, machine-learning-driven system that detects and blocks fraudulent transactions in real-time with high accuracy and minimal disruption to legitimate users.

Technical Context

-What is Fraud Detection?

-A data-driven process that uses machine learning algorithms to analyze transaction patterns, identify anomalies, and stop unauthorized or deceptive financial activities.

-Why it Matters:

-Saves Money: Prevents direct financial losses and chargeback fees.

-Builds Trust: Protects customers without ruining their shopping experience.

-Cuts Costs: Automates security, reducing the need for manual reviews.

Dataset Description

-Source: Kaggle (look at https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud/data)

-The dataset contains transactions made by credit cards in September 2013 by European cardholders. This dataset presents transactions that occurred in two days.

- We have 492 frauds out of 284,807 transactions.

-We have (30 features: V1, V2, … V28 are the principal components obtained with PCA, the only features that have not been transformed with PCA are 'Time' and 'Amount', more details on kaggle link)

-Target 'Class' is the response variable and it takes the value 1 in case of fraud and 0 otherwise.

-I used 170,884 samples for training(train.csv)

-I used the rest for both (val.csv and test.csv)

Note: this splitting of data into files was made by someone from whom I have the right to use these files.

Exploratory Data Analysis (EDA)

-You can find EDA(including cleaning, feature engineering) insights in the GitHub repo at reports/read my insights.docx

Data Preprocessing

-I used **RobustScaler** over minmax or z-score with column **Amount**

because it is robust to outliers (extremely high/low values), which are naturally frequent in fraud data patterns.

-I used SimpleImputer with all columns to replace missing feature entries with the **`median`** strategy to prevent machine learning estimators from crashing due to null values.

Modeling

1. Baseline Model: Logistic Regression

- **Status:** Underfits the data (indicates a more complex model is needed).

- **Validation Dataset Performance:**
  - **F1-Score:** 0.09195
  - **Precision:** 0.05
  - **Recall:** 0.89

2. Advanced Model: RandomForestClassifier

- **Configuration:** `class_weight: 'balanced'`
- **Status:** Really good performance, handles data complexity well.
- **Validation Dataset Performance:**
  - **F1-Score:** 0.84706
  - **Precision:** 0.81
  - **Recall:** 0.87
- **Action taken:** Saved this model as `.pkl` for final testing.

3. Final Evaluation: Test Dataset (unseen)

- **Status:** Ensures good generalization.
- **Test Data Results (`test.csv`):**
  - **F1-Score:** 0.83582
  - **Precision:** 0.81
  - **Recall:** 0.87

Project pipeline

1. Training Pipeline

1. **Data Ingestion:** Load the training dataset.
2. **Feature Engineering:** Inject newly created features/attributes.
3. **Data Cleaning:** Drop duplicate rows.
4. **Target Splitting:** Separate features (`X`) from the target label (`y`).
5. **Data Preprocessing:** Learn and apply scaling/imputation parameters (`Fit & Transform`), then save the preprocessor.
6. **Model Training:** Train the machine learning model and save it.

2. Validation Pipeline

1. **Data Ingestion:** Load the validation dataset.

2. **Feature Engineering:** Inject the exact same new features.
3. **Data Cleaning:** Pass the data as-is (no duplicate removal).
4. **Target Splitting:** Separate features (`X`) from the target label (`y`).
5. **Data Preprocessing:** Apply the saved training scaling parameters only (`Transform`).
6. **Model Evaluation:** Test model accuracy to tune performance.

3. Testing Pipeline

1. **Data Ingestion:** Load the new independent testing file (`test.csv`).
2. **Feature Engineering:** Generate identical custom features.
3. **Target Splitting:** Separate features (`X`) from the target label (`y`).
4. **Preprocessor Loading:** Load the saved preprocessor file and apply modifications (`Transform`).
5. **Prediction & Evaluation:** Load the finalized model, **execute the final predictions**, and output the success rate.

Future Improvements

-FastAPI deployment.

-Docker containerization.